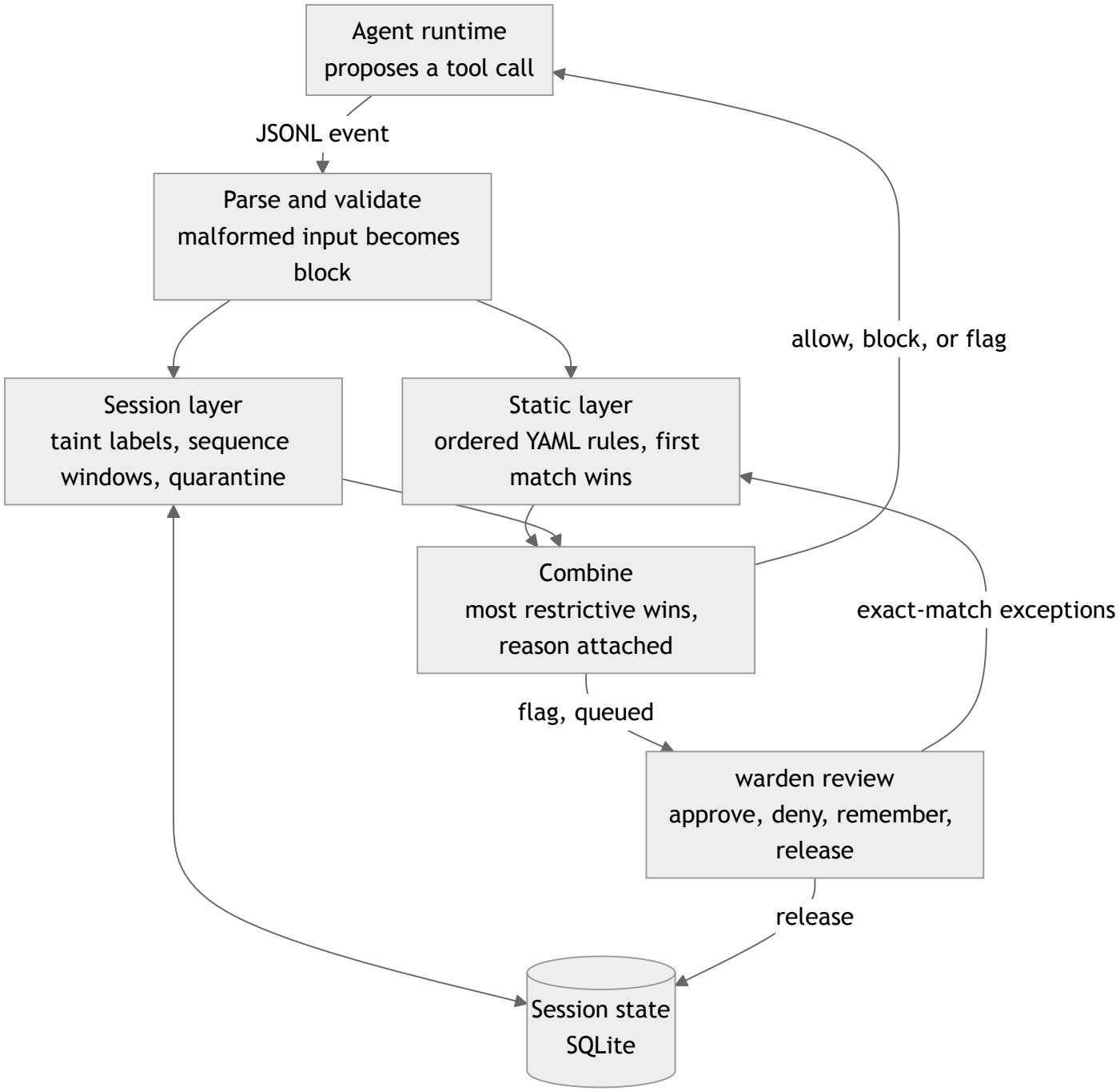
warden: design choices and known limitations

warden is a policy-enforcing middleware layer between an LLM agent and its tools. It reads a stream of proposed tool calls and returns allow, block, or flag for each, with a reason a human can act on. It is a deterministic reference monitor with no LLM in the decision path: same events, same policy, same session state, same decisions. A firewall has to be fast, explainable, and testable, and a probabilistic component in the hot path forfeits all three. I took the same stance building deterministic policy gating for infrastructure pull requests: the gate itself must be boring and auditable.

How a call is decided



One event in, one decision out. The engine is a pure library; the CLI loads a session's state, decides, and writes it back. Every decision carries the rule that decided it and every rule that matched.

Design choices

Two layers, combined asymmetrically. Static rules are first-match-wins, classic ACL semantics, so order is priority and targeted exceptions stay expressible. Across layers the most restrictive verdict wins (block > flag > allow), because a call that static policy allows must still be blockable as an exfiltration sink in a tainted session. Every rule that fired is recorded on the decision, so the reader sees which layer decided and why.

Sequences as taint plus windowed rules, both in config. The exfiltration chain (read customer data, then POST it) is invisible per call. A configured source read labels the session with provenance; configured sinks escalate while the label is present, naming the tainting call. This forced a split between two classes of sensitive data: never-readable paths (SSH keys, `.env`) are statically blocked, while readable-but-not-exfiltrable paths (PII the agent legitimately processes) are allowed and taint the session. Conflating them makes taint dead code, since a blocked read never taints. The never-readable rule sits first in the list and is mirrored for shell arguments, because first match wins and the shell can otherwise read what the file tool cannot. Windowed rules cover the rest: write-then-execute compares canonicalized path tokens in the shell command against recently written paths, and probing counts blocks in a window.

Sink severity is per tool. The binding constraint is "without breaking legitimate agent workflows," and the usual reason an agent reads PII is to process it locally or enrich it via an API. So `http.post` after a PII read blocks (direct egress), while `http.get` and `shell.exec` flag: session-level taint cannot separate processing from exfiltration, so a human decides.

A sequence rule indicts a call or a session. Write-then-execute blocks the executing call. Probing quarantines the session, because the tripping calls are already blocked and a per-call flag would never surface on them. Quarantine downgrades every allow to flag until a human releases it. Those flags are audited but not queued individually: a hijacked agent then emits hundreds of calls, and the session is the review unit.

Flag is a real state with a real resolution path. Flags persist in SQLite; `warden review` lists, shows, approves, denies, and releases. Approving with `--remember` mints an exact-match exception (tool plus exact argument values), evaluated before the main rules. Close variants re-flag on purpose: a human approved one call, not a pattern. Minting is refused while the session is quarantined, since the allow would be silently outranked until release.

Fail closed, persist state. Malformed lines, invalid events, unknown tools, oversized or non-UTF-8 input: every failure path degrades to block with a reason, never to allow, and the stream continues. Shell commands are split at control operators: an allow must cover every segment as a simple command, while block, flag, and sequence steps fire on any segment, so `ls ; curl` cannot ride an `ls` allow and a chained execute still trips write-then-execute. An invalid policy refuses to start. Session state lives in SQLite, so a one-shot check honors an earlier quarantine and release has an observable effect; the in-memory working set is LRU-bounded with eviction to the store, so a session-id spray cannot exhaust memory or lose a label. A reachability lint warns when a taint source or sequence step can never execute under the static rules, because design review found that bug four times.

Known limitations

1. **Session-level taint is coarse.** After one sensitive read, every outbound call escalates, innocent ones included. Mitigation: per-argument provenance and declassification rules (an approved redaction step clears the label).
2. **Argument matching is bypassable by indirection**: base64 in shell arguments, URL redirects, or laundering a secret through an allowed path. Percent-encoded paths such as `%2e%2e` are matched literally, since a filesystem does not decode them; a runtime that does would need to decode before proposing. Production needs canonicalization and enforcement at the execution boundary.
3. **warden decides; it does not enforce.** The CLI is one adapter over a pure engine; in production the same engine sits inside the tool-execution gateway, and a runtime that ignores decisions gets no protection.
4. **No cross-session correlation.** Splitting a chain across two sessions evades the session layer.
5. **Time-of-check to time-of-use.** A path can change between decision and execution (symlink swap). Inherent to judging proposals.
6. **No session lifecycle.** Without a session-end event, persisted state lives until pruned; memory is bounded, the store is not. Mitigation: an end event or idle TTL, with expiry logged so a timed-out quarantine is never silent.
7. **Single-writer assumption.** State updates are read-modify-write without locking; concurrent writers on one session can lose a label or a quarantine. Fine for a single-operator CLI; production needs transactional updates.